

OPTION B INSTITUTE FAMILY

M02 - SQL FOUNDATIONS

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

14 canonical lessons | full 16-stage teacher contract | protected attempt-first review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE02-01 - Database, table, row, column and query mental model	5
DE02-02 - SELECT, aliases and LIMIT	7
DE02-03 - WHERE and comparison operators	9
DE02-04 - AND, OR, NOT and parentheses	11
DE02-05 - ORDER BY, DISTINCT and result inspection	13
DE02-06 - NULL, IS NULL and COALESCE	15
DE02-07 - Arithmetic expressions and safe calculations	17
DE02-08 - Aggregate functions	19
DE02-09 - GROUP BY and grouped grain	21
DE02-10 - HAVING after aggregation	23
DE02-11 - CASE for business categories	25
DE02-12 - String and date basics	27
DE02-13 - Simple subqueries and IN/EXISTS awareness	29
DE02-14 - Debugging and validation workflow	31
Module mastery checklist + revision quick reference	33

Prerequisite / tool map

Translate business questions into correct foundational SQL and validate the result.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE02-01 - Database, table, row, column and query mental model

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Install/verify MySQL 8+ and MySQL Workbench, load the course database, and explain the difference between a database, table, row, column, query and result grid.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

A database stores structured objects. A table is a named set of rows and columns.

5. New words before we use them

A database stores structured objects. A table is a named set of rows and columns. In this lab, sales_orders has one row per order.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 0:52-11:19

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

USE stage02_retail_lab; SELECT DATABASE() AS current_database; SELECT COUNT(*) AS total_rows FROM sales_orders; Run the setup script. Confirm the active database is stage02_retail_lab and the table has exactly 30 rows. Open DESCRIBE sales_orders and point to order_id, qty, status and sales_rep.

8. Expected result / what you should see

Database = stage02_retail_lab; row count = 30; order_id is the primary key; sales_rep can be NULL.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the database/table is missing, rerun only the setup script and confirm no error occurred before writing learning queries. Do not install WSL for this module; M02 uses MySQL on Windows.

11. If you get stuck

If the database/table is missing, rerun only the setup script and confirm no error occurred before writing learning queries. Do not install WSL for this module; M02 uses MySQL on Windows.

12. Small independent task

With the video closed, open a new query tab and return order_id, region and product for any 5 rows. In a SQL comment, state the result grain.

13. Validate

Exactly 3 selected columns and 5 rows. Grain statement says one result row represents one order from sales_orders. Database = stage02_retail_lab; row count = 30; order_id is the primary key; sales_rep can be NULL.

14. Teach it back

Explain database vs table vs query vs result grid in 45-60 seconds.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-02

DE02-02 - SELECT, aliases and LIMIT

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose output columns, calculate a simple expression, rename output with AS, and limit the number of returned rows.

3. Prerequisite / starting state

Prerequisite: DE02-01 mastered with saved evidence.

4. Picture it in your head

SELECT controls the projection: which columns or expressions appear in the result. AS gives an output label.

5. New words before we use them

SELECT controls the projection: which columns or expressions appear in the result. AS gives an output label. LIMIT reduces the result rows returned after the query logic has run; it does not delete source rows.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 11:19-21:47 + 47:41-51:10

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id, qty, unit_price, qty * unit_price AS gross_sales FROM sales_orders LIMIT 10; Return order_id, qty, unit_price and gross_sales for 10 rows. Rename product as item_name in a second query.

8. Expected result / what you should see

O1001 gross_sales = 1700; O1002 = 1350. First query returns 10 rows and 4 columns.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the alias is unavailable in the output, check AS spelling and whether the expression has a comma before it. If LIMIT is ignored, confirm it is at the end of the SELECT statement.

11. If you get stuck

If the alias is unavailable in the output, check AS spelling and whether the expression has a comma before it. If LIMIT is ignored, confirm it is at the end of the SELECT statement.

12. Small independent task

Return order_id, order_date and product for every row, then create a separate query returning 7 rows with product renamed item_name.

13. Validate

First query returns 30 rows / 3 columns. Second returns 7 rows and has a column named item_name. O1001 gross_sales = 1700; O1002 = 1350. First query returns 10 rows and 4 columns.

14. Teach it back

Explain why SELECT and LIMIT change the result but not the stored table.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-03

DE02-03 - WHERE and comparison operators

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Translate a single business condition into WHERE filters using =, <>, >, >=, <, <=, BETWEEN and IN.

3. Prerequisite / starting state

Prerequisite: DE02-02 mastered with saved evidence.

4. Picture it in your head

WHERE decides which source rows are eligible before grouping. Every returned row must satisfy the condition.

5. New words before we use them

WHERE decides which source rows are eligible before grouping. Every returned row must satisfy the condition. A filter should be readable as the business rule you were given.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 21:47–33:54

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id, region, product, qty, status FROM sales_orders WHERE status = 'Completed' AND qty >= 2; Run the Completed AND qty >= 2 filter and a separate COUNT(*) using the identical WHERE clause.

8. Expected result / what you should see

22 rows; no Pending or Cancelled rows survive.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the count differs, read the English rule and test each predicate separately. Do not edit several clauses at once.

11. If you get stuck

If the count differs, read the English rule and test each predicate separately. Do not edit several clauses at once.

12. Small independent task

Return orders dated 2026-07-10 through 2026-07-12 inclusive, then return only West orders with unit_price >= 850.

13. Validate

Date-range result = 6 rows. West/high-price result contains only West and unit_price >= 850. 22 rows; no Pending or Cancelled rows survive.

14. Teach it back

Explain WHERE as an eligibility rule, not just syntax.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-04

DE02-04 - AND, OR, NOT and parentheses

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Combine conditions safely and use parentheses so Boolean logic matches the business request.

3. Prerequisite / starting state

Prerequisite: DE02-03 mastered with saved evidence.

4. Picture it in your head

AND requires both conditions. OR allows either.

5. New words before we use them

AND requires both conditions. OR allows either. NOT reverses a condition.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 21:47–33:54

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id, region, status FROM sales_orders WHERE status='Completed' AND (region='East' OR region='North'); Return Completed orders from East or North with explicit parentheses. Count the rows separately.

8. Expected result / what you should see

13 rows. Regions are only East/North and every status is Completed.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a North Pending row appears, inspect the parentheses. Translate the WHERE clause back into English before rerunning.

11. If you get stuck

If a North Pending row appears, inspect the parentheses. Translate the WHERE clause back into English before rerunning.

12. Small independent task

Return non-Cancelled orders from East or North. Then write a logically equivalent version using NOT status = 'Cancelled'.

13. Validate

Both independent queries return the same order_ids. Save a set/count comparison. 13 rows. Regions are only East/North and every status is Completed.

14. Teach it back

Explain why A AND (B OR C) is different from (A AND B) OR C.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-05

DE02-05 - ORDER BY, DISTINCT and result inspection

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Remove repeated result values with DISTINCT, sort intentionally, and use top-N inspection without confusing sorting with filtering.

3. Prerequisite / starting state

Prerequisite: DE02-04 mastered with saved evidence.

4. Picture it in your head

DISTINCT removes duplicate combinations from the query result. ORDER BY changes sequence.

5. New words before we use them

DISTINCT removes duplicate combinations from the query result. ORDER BY changes sequence. LIMIT keeps only the first N rows after sorting.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 11:19–21:47 + 47:41–51:10

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT DISTINCT region FROM sales_orders ORDER BY region; SELECT order_id, product, qty*unit_price AS gross_sales FROM sales_orders ORDER BY gross_sales DESC LIMIT 5; List unique regions alphabetically. In a separate query return the five highest gross-sales orders.

8. Expected result / what you should see

Regions = East, North, South, West. Highest gross order = O1020 at 3600.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

For a top-N question, sort DESC before LIMIT. If DISTINCT returns more values than expected, inspect all selected columns: DISTINCT applies to the whole selected combination.

11. If you get stuck

For a top-N question, sort DESC before LIMIT. If DISTINCT returns more values than expected, inspect all selected columns: DISTINCT applies to the whole selected combination.

12. Small independent task

Return distinct payment methods alphabetically. Then return the three largest-quantity orders with a deterministic tie-break on order_id.

13. Validate

Exactly Card, Cash, UPI. Top qty begins O1021=10, O1026=9, O1007=8. Regions = East, North, South, West. Highest gross order = O1020 at 3600.

14. Teach it back

Explain DISTINCT on one column versus DISTINCT on two columns.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-06

DE02-06 - NULL, IS NULL and COALESCE

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Find missing values correctly, explain COUNT(*) versus COUNT(column), and use COALESCE only when a display/default rule is justified.

3. Prerequisite / starting state

Prerequisite: DE02-05 mastered with saved evidence.

4. Picture it in your head

NULL means no value is present; it is not zero and not empty text. Use IS NULL / IS NOT NULL rather than = NULL.

5. New words before we use them

NULL means no value is present; it is not zero and not empty text. Use IS NULL / IS NOT NULL rather than = NULL. COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 10:16:15 — Functions: Date, Text & NULL (focus NULL/COALESCE)

How to use it: Use the chapter like a paid-course class; then close it and complete the course NULL/COALESCE SQL task.

Note: Current 2026 DE-focused SQL teaching.

[Open source](#)

7. Follow with me once

SELECT COUNT(*) AS rows_all, COUNT(sales_rep) AS rows_with_rep FROM sales_orders; SELECT order_id, COALESCE(sales_rep, 'Unassigned') AS rep_display FROM sales_orders WHERE sales_rep IS NULL; Compare COUNT(*) with COUNT(sales_rep), find the NULL row, and display Unassigned with COALESCE without updating the table.

8. Expected result / what you should see

30 total rows; 29 non-NULL sales_rep; O1030 is the one NULL sales_rep row.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If = NULL returns nothing, replace it with IS NULL. Do not use COALESCE to pretend missing source data is known; here it is a presentation label only.

11. If you get stuck

If = NULL returns nothing, replace it with IS NULL. Do not use COALESCE to pretend missing source data is known; here it is a presentation label only.

12. Small independent task

Return all rows with sales_rep IS NOT NULL and prove their count. Then show every order with a rep_label that uses Unassigned only for display.

13. Validate

IS NOT NULL count = 29. Source sales_rep for O1030 remains NULL. 30 total rows; 29 non-NULL sales_rep; O1030 is the one NULL sales_rep row.

14. Teach it back

Explain NULL, IS NULL, COUNT(column), and COALESCE with O1030.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-07

DE02-07 - Arithmetic expressions and safe calculations

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build auditable gross/net calculations from source columns and validate them manually.

3. Prerequisite / starting state

Prerequisite: DE02-06 mastered with saved evidence.

4. Picture it in your head

A calculated result should show its business formula. $\text{gross_sales} = \text{qty} * \text{unit_price}$.

5. New words before we use them

A calculated result should show its business formula. $\text{gross_sales} = \text{qty} * \text{unit_price}$. $\text{net_sales} = \text{gross_sales} * (1 - \text{discount_pct})$.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 1:39:28 — Operators & Functions

How to use it: Watch the operators/functions chapter, type examples, then solve the course safe-calculation task with the video closed.

Note: Current 2026 long-form DE bootcamp.

[Open source](#)

7. Follow with me once

`SELECT order_id, qty, unit_price, discount_pct, qty*unit_price AS gross_sales, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders;` Calculate gross_sales and net_sales for all orders. Manually verify O1002.

8. Expected result / what you should see

O1002 gross = 1350; net = 1282.50.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If net is larger than gross for a positive discount, inspect the sign. If decimals look strange, round for presentation only after the underlying calculation is correct.

11. If you get stuck

If net is larger than gross for a positive discount, inspect the sign. If decimals look strange, round for presentation only after the underlying calculation is correct.

12. Small independent task

Calculate $\text{discount_value} = \text{gross_sales} - \text{net_sales}$ and validate two orders manually.

13. Validate

For every row with $\text{discount_pct} > 0$, $\text{discount_value} > 0$ and $\text{net_sales} \leq \text{gross_sales}$. Save two manual checks. O1002 gross = 1350; net = 1282.50.

14. Teach it back

Explain the difference between stored source fields and derived query expressions.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-08

DE02-08 - Aggregate functions

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Summarize eligible rows with COUNT, SUM, AVG, MIN and MAX and explain what NULL does to common aggregates.

3. Prerequisite / starting state

Prerequisite: DE02-07 mastered with saved evidence.

4. Picture it in your head

Without GROUP BY, an aggregate summarizes the filtered input into one result row. COUNT(*) counts rows, while COUNT(column) ignores NULL values.

5. New words before we use them

Without GROUP BY, an aggregate summarizes the filtered input into one result row. COUNT(*) counts rows, while COUNT(column) ignores NULL values. SUM/AVG/MIN/MAX operate on the values that survive filtering.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 33:54–44:18

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT COUNT(*) AS order_rows, SUM(qty) AS total_qty, AVG(unit_price) AS avg_unit_price, MIN(unit_price) AS min_price, MAX(unit_price) AS max_price, SUM(qty*unit_price) AS gross_sales FROM sales_orders; Return row count, total qty, average price, min/max price and gross sales for all rows.

8. Expected result / what you should see

30 rows; total qty 105; average unit_price about 666.33; gross sales 50000.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If an aggregate is unexpectedly high/low, first inspect the eligible row count and whether the filter matches the business rule.

11. If you get stuck

If an aggregate is unexpectedly high/low, first inspect the eligible row count and whether the filter matches the business rule.

12. Small independent task

For Completed orders only, return order count, total qty and total net_sales in one summary row.

13. Validate

Completed order count = 27; total qty = 100; total net_sales = 42984.50. 30 rows; total qty 105; average unit_price about 666.33; gross sales 50000.

14. Teach it back

Explain why a one-row aggregate has a different result grain from the source table.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-09

DE02-09 - GROUP BY and grouped grain

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Create one summary row per group, state output grain first, and reconcile grouped totals to an overall control.

3. Prerequisite / starting state

Prerequisite: DE02-08 mastered with saved evidence.

4. Picture it in your head

GROUP BY changes result grain. If you GROUP BY region, one output row represents one region.

5. New words before we use them

GROUP BY changes result grain. If you GROUP BY region, one output row represents one region. Selected non-aggregate columns must belong to the grouping grain.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 33:54–44:18

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT region, COUNT(*) AS orders, SUM(qty) AS total_qty, SUM(qty*unit_price) AS gross_sales FROM sales_orders WHERE status='Completed' GROUP BY region ORDER BY region; Build Completed order count and gross sales by region.

8. Expected result / what you should see

East 8 / 12800; North 5 / 9310; South 7 / 12010; West 7 / 10930.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If grouped totals do not reconcile, run the same measure with the same WHERE filter without GROUP BY. Do not trust plausible-looking group rows.

11. If you get stuck

If grouped totals do not reconcile, run the same measure with the same WHERE filter without GROUP BY. Do not trust plausible-looking group rows.

12. Small independent task

Build Completed order count, qty and net_sales by payment_method and state the result grain.

13. Validate

3 result rows; grouped net total reconciles to 42984.50. East 8 / 12800; North 5 / 9310; South 7 / 12010; West 7 / 10930.

14. Teach it back

Explain how GROUP BY changes grain from one row/order to one row/group.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-10

DE02-10 - HAVING after aggregation

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use WHERE for source-row eligibility and HAVING for conditions on grouped/aggregate results.

3. Prerequisite / starting state

Prerequisite: DE02-09 mastered with saved evidence.

4. Picture it in your head

WHERE runs before aggregation and filters source rows. HAVING filters groups after GROUP BY.

5. New words before we use them

WHERE runs before aggregation and filters source rows. HAVING filters groups after GROUP BY. A condition such as status=Completed belongs in WHERE; a condition such as SUM(qty)>=25 belongs in HAVING.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 44:18–47:41

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT region, SUM(qty) AS total_qty FROM sales_orders GROUP BY region HAVING SUM(qty) >= 25 ORDER BY total_qty DESC; Group all rows by region and keep groups with SUM(qty) >= 25.

8. Expected result / what you should see

East 29, North 27, West 25; South is excluded.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you put SUM(qty) in WHERE, move the aggregate condition to HAVING. If business eligibility is wrong, inspect WHERE separately.

11. If you get stuck

If you put SUM(qty) in WHERE, move the aggregate condition to HAVING. If business eligibility is wrong, inspect WHERE separately.

12. Small independent task

For Completed orders only, return payment methods whose total qty is at least 25.

13. Validate

Cash 39 and UPI 39 qualify; Card 22 does not. East 29, North 27, West 25; South is excluded.

14. Teach it back

Say when WHERE happens and when HAVING happens, using one example of each.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-11

DE02-11 - CASE for business categories

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Translate business rules into ordered CASE branches and validate bucket coverage.

3. Prerequisite / starting state

Prerequisite: DE02-10 mastered with saved evidence.

4. Picture it in your head

CASE tests WHEN branches from top to bottom and returns the first matching result. For overlapping thresholds, put the highest/more restrictive rule first.

5. New words before we use them

CASE tests WHEN branches from top to bottom and returns the first matching result. For overlapping thresholds, put the highest/more restrictive rule first.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 1:26:39–1:35:16

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
SELECT order_id, qty*unit_price AS gross_sales, CASE WHEN qty*unit_price >= 2500 THEN 'High'
WHEN qty*unit_price >= 1500 THEN 'Medium' ELSE 'Small' END AS order_size FROM sales_orders;
Bucket all orders into High >=2500, Medium >=1500, otherwise Small, then count each bucket.
```

8. Expected result / what you should see

High 3; Medium 14; Small 13.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If High is empty, inspect branch order. If counts do not sum to 30, inspect missing ELSE or filter logic.

11. If you get stuck

If High is empty, inspect branch order. If counts do not sum to 30, inspect missing ELSE or filter logic.

12. Small independent task

Create fulfillment_group: Completed -> Closed; Pending -> Open; otherwise Other. Count each group.

13. Validate

Counts across the three groups sum to 30 and source status values remain unchanged. High 3; Medium 14; Small 13.

14. Teach it back

Explain why CASE is derived result logic and not an UPDATE.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-12

DE02-12 - String and date basics

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use basic text cleanup/display functions and extract date parts while preserving source data.

3. Prerequisite / starting state

Prerequisite: DE02-11 mastered with saved evidence.

4. Picture it in your head

TRIM removes surrounding whitespace from a result expression, UPPER changes display casing, and YEAR/MONTH extract calendar components in MySQL. These query expressions do not update source values.

5. New words before we use them

TRIM removes surrounding whitespace from a result expression, UPPER changes display casing, and YEAR/MONTH extract calendar components in MySQL. These query expressions do not update source values.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 1:15:10–1:26:39

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id, UPPER(TRIM(region)) AS region_clean, YEAR(order_date) AS order_year, MONTH(order_date) AS order_month FROM sales_orders WHERE order_date BETWEEN '2026-07-10' AND '2026-07-12'; Run the date-range/text query for July 10 through July 12 inclusive.

8. Expected result / what you should see

6 rows; every order_year = 2026 and order_month = 7.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If dates are missing, check endpoints and data type. If text cleanup changes the business meaning, do not treat it as a safe standardization without a rule.

11. If you get stuck

If dates are missing, check endpoints and data type. If text cleanup changes the business meaning, do not treat it as a safe standardization without a rule.

12. Small independent task

Return product labels as UPPER(product) and extract day-of-month for orders on July 13-15. Keep original product beside the transformed value.

13. Validate

Only July 13, 14 and 15 rows appear; transformed output is traceable to original text. 6 rows; every order_year = 2026 and order_month = 7.

14. Teach it back

Explain query-time transformation versus changing stored source data.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-13

DE02-13 - Simple subqueries and IN/EXISTS awareness

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use a simple scalar subquery and recognize IN/EXISTS as ways to filter based on another query result.

3. Prerequisite / starting state

Prerequisite: DE02-12 mastered with saved evidence.

4. Picture it in your head

A subquery is a query nested inside another query. A scalar subquery returns one value, such as an average.

5. New words before we use them

A subquery is a query nested inside another query. A scalar subquery returns one value, such as an average. IN compares a value against a set returned by a subquery.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 1:35:16–1:45:58

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders WHERE status='Completed' AND qty*unit_price*(1-discount_pct) > (SELECT AVG(qty*unit_price*(1-discount_pct)) FROM sales_orders WHERE status='Completed') ORDER BY net_sales DESC; Find Completed orders whose net_sales is above the average Completed net_sales. Run the inner average query alone first.

8. Expected result / what you should see

Average Completed net_sales is about 1592.02; 13 orders are above it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the outer result is wrong, run the inner query independently and verify it returns one value. Keep the same Completed eligibility in both places.

11. If you get stuck

If the outer result is wrong, run the inner query independently and verify it returns one value. Keep the same Completed eligibility in both places.

12. Small independent task

Use IN with a subquery to return orders whose region belongs to a region whose all-status SUM(qty) is at least 25. State what the inner query returns.

13. Validate

The inner set is East, North, West; no South order appears in the outer result. Average Completed net_sales is about 1592.02; 13 orders are above it.

14. Teach it back

Explain scalar subquery versus IN in simple English; give one reason to validate the inner query separately.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: DE02-14

DE02-14 - Debugging and validation workflow

1. Why it matters

This matters because later SQL Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Debug syntax/logic problems systematically and prove a result with independent counts, grain checks and reconciliations rather than trusting a green Run button.

3. Prerequisite / starting state

Prerequisite: DE02-13 mastered with saved evidence.

4. Picture it in your head

Debugging has two branches: the query fails to run, or it runs but the answer is wrong. For syntax errors, read the exact error and reduce the query.

5. New words before we use them

Debugging has two branches: the query fails to run, or it runs but the answer is wrong. For syntax errors, read the exact error and reduce the query. For wrong results, verify source grain, filters, row counts and aggregates.

6. Useful video / source

Source: Course research bundle — SQL troubleshooting + wrong-result validation

Assigned scope: Microsoft troubleshooting process video + Metabase duplicate/wrong-result logic guide; then course-owned validation lab

How to use it: Use the short troubleshooting video for the systematic debugging mindset, then the duplicate-result guide for logic validation. Close sources and debug the course query independently.

Note: No single external video exactly teaches our full validation workflow, so the bundle combines two narrow sources and keeps the course lab authoritative.

[Open source](#)

7. Follow with me once

-- Wrong business rule: Pending rows are accidentally included SELECT region, SUM(qty*unit_price) AS gross_sales FROM sales_orders WHERE status <> 'Cancelled' GROUP BY region; -- Control SELECT COUNT(*) FROM sales_orders WHERE status='Completed'; Run the wrong-business-rule query, compare its eligibility count with Completed-only count, repair the filter, then reconcile grouped gross_sales to a separate Completed overall total.

8. Expected result / what you should see

Completed-only count = 27. Completed grouped gross total reconciles to 45050. A query that includes Pending is not equivalent even if the numbers look plausible.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Do not randomly edit SQL. Record expected result, actual result, exact error/wrong signal, one hypothesis, one change and rerun evidence. Preserve the failed attempt.

11. If you get stuck

Do not randomly edit SQL. Record expected result, actual result, exact error/wrong signal, one hypothesis, one change and rerun evidence. Preserve the failed attempt.

12. Small independent task

Take a deliberately wrong query that uses OR without parentheses, diagnose why a Pending row leaks into the output, repair it, and save before/after row counts plus the corrected English rule.

13. Validate

The corrected query has the intended status and regions only. A separate COUNT/control query supports the result. Completed-only count = 27. Completed grouped gross total reconciles to 45050. A query that includes Pending is not equivalent even if the numbers look plausible.

14. Teach it back

Explain the difference between a syntax bug, a logic bug and a validation check. Give one example of each.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save your SQL, observed output/count/control, one error or uncertainty, confidence 0-5, and a 30-60 second own-words explanation. If review was needed, mastery requires a fresh changed retry with the review closed. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.